

Informe laboratorio 1

1. Link del repositorio

<https://github.com/CamiR24/Lab1-DeepLearning.git>

2. Optimizadores

nn.Linear

Implementa una transformación lineal entre una capa y la siguiente. Cada neurona recibe todas las salidas de la capa anterior y calcula una combinación lineal de ellas, aprendiendo un conjunto de pesos y un sesgo durante el entrenamiento. Es la capa fundamental para construir un perceptrón multicapa.

Parámetros principales:

- `in_features`: número de características de entrada.
- `out_features`: número de neuronas de salida.
- `bias`: indica si la capa utiliza un sesgo, es `True` por defecto.

Funciones de activación

Las funciones de activación introducen no linealidad en la red, permitiendo que el modelo aprenda relaciones complejas entre las variables.

nn.ReLU

Devuelve el valor de entrada cuando es positivo y cero cuando es negativo. Es rápida computacionalmente, reduce el problema del gradiente desvanecido, es la función de activación más utilizada en redes neuronales profundas.

nn.LeakyReLU

Es una variante de ReLU que permite un pequeño gradiente cuando la entrada es negativa. Evita que algunas neuronas dejen de aprender, mantiene un pequeño flujo de gradiente para valores negativos.

nn.Tanh

La función tangente hiperbólica transforma los datos al intervalo: $[-1,1]$. Produce salidas centradas en cero, funciona bien cuando los datos fueron estandarizados. Por otro lado, en redes profundas puede sufrir el problema del gradiente desvanecido.

nn.Dropout

Técnica de regularización que durante el entrenamiento desactiva aleatoriamente un porcentaje de neuronas. Busca evitar que la red memorice los datos de entrenamiento y mejorar la capacidad de generalización.

Parámetro principal:

- `p`: probabilidad de desactivar una neurona.

nn.BatchNorm1d

Normaliza las activaciones de cada mini-batch durante el entrenamiento. Esto permite acelerar la convergencia, estabilizar el entrenamiento, permitir tasas de aprendizaje más altas y reducir el sobreajuste.

Parámetro principal:

- `num_features`: número de características de entrada que serán normalizadas.

funciones de pérdida de regresión

Mide qué tan lejos están las predicciones del modelo respecto a los valores reales.

nn.MSELoss

Calcula el Error Cuadrático Medio (Mean Squared Error). Penaliza fuertemente los errores grandes, es la función más utilizada en problemas de regresión.

nn.L1Loss

Calcula el Error Absoluto Medio (Mean Absolute Error). Es más robusta frente a valores atípicos, todos los errores tienen el mismo peso.

nn.SmoothL1Loss

Combina las ventajas de MSE y MAE, porque para errores pequeños se comporta como MSE y para errores grandes se comporta como MAE. Tiene menor sensibilidad a los outliers y hace que el entrenamiento sea más estable.

Optimizadores

Actualizan los pesos de la red usando el gradiente calculado durante backpropagation. Buscan minimizar la función de pérdida para que el modelo aprenda la relación entre las variables de entrada y la variable objetivo.

Parámetros importantes:

- `lr` (learning rate): Es la tasa de aprendizaje, determina el tamaño de los pasos que el optimizador da al actualizar los pesos (si es muy grande, el

entrenamiento puede volverse inestable y no converger y si es muy pequeña, el entrenamiento es muy lento)

- **weight_decay**: Aplica regularización L2. Busca penalizar pesos muy grandes para reducir el sobreajuste y mejorar la capacidad de generalización.

SGD (Stochastic Gradient Descent)

Actualiza los pesos utilizando el gradiente calculado en cada iteración. Es el optimizador más simple, consume poca memoria, puede tardar más en converger y requiere elegir cuidadosamente el learning rate.

Adam (Adaptive Moment Estimation)

Adapta automáticamente la tasa de aprendizaje para cada parámetro usando estimaciones del primer y segundo momento del gradiente. Converge rápidamente, requiere poca afinación de hiperparámetros, es uno de los optimizadores más usados en redes neuronales.

RMSprop

Ajusta la tasa de aprendizaje de cada parámetro con el promedio móvil de los gradientes recientes. Funciona bien cuando los gradientes cambian constantemente, es más estable que SGD y generalmente converge más rápido que SGD, aunque es superado por Adam en muchas aplicaciones.

3. Tabla de resultados

	Modelo	Arquitectura	Activación	Optimizador	Learning Rate	Batch Size	Dropout	Weight Decay	Best Epoch	MSE	MAE	RMSE
9	10	[128, 64]	relu	adam	0.001	64	0.2	0.001	100	2.811240e+10	147341.984375	167667.517093
8	9	[128, 64]	relu	adam	0.001	64	0.2	0.000	100	2.851067e+10	148501.140625	168851.043657
5	6	[64, 32]	leakyrelu	adam	0.001	64	0.2	0.001	100	4.013752e+10	175815.937500	200343.512997
1	2	[64, 32]	relu	adam	0.001	64	0.2	0.001	100	4.015640e+10	175881.203125	200390.612475
0	1	[64, 32]	relu	adam	0.001	64	0.2	0.000	100	4.048928e+10	176676.703125	201219.470708
4	5	[64, 32]	leakyrelu	adam	0.001	64	0.2	0.000	100	4.080450e+10	177373.593750	202001.226135
7	8	[64, 32]	leakyrelu	rmsprop	0.001	64	0.2	0.001	100	4.202149e+10	179905.546875	204991.430533
2	3	[64, 32]	relu	rmsprop	0.001	64	0.2	0.000	100	4.238696e+10	180681.734375	205880.937398
6	7	[64, 32]	leakyrelu	rmsprop	0.001	64	0.2	0.000	100	4.251386e+10	180847.781250	206188.900419
3	4	[64, 32]	relu	rmsprop	0.001	64	0.2	0.001	100	4.255096e+10	180824.578125	206278.840718

4. Análisis de resultados

¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo en las métricas de validación? ¿Y el mayor impacto negativo?

El cambio de hiperparámetro que tuvo el mayor impacto positivo fue incrementar la arquitectura del MLP de dos capas ocultas de 64 y 32 neuronas a una arquitectura de 128 y 64 neuronas, manteniendo el optimizador Adam y la función de activación ReLU. Además, incorporar una regularización L2 permitió obtener la mejor configuración.

Por otro lado, el mayor impacto negativo se observó al usar el optimizador RMSprop, ya que todas las configuraciones que lo emplearon obtuvieron errores superiores a las

obtenidas con Adam. Para este conjunto de datos, Adam fue más eficiente para optimizar los pesos.

¿Observó overfitting o underfitting en alguna de sus iteraciones? ¿Cómo lo identificó y qué hizo para mitigarlo?

No se observó overfitting durante las 100 épocas de entrenamiento. Las curvas de pérdida de entrenamiento y validación disminuyeron de forma paralela y permanecieron muy cercanas entre sí, sin que la pérdida de validación comenzara a aumentar mientras la pérdida de entrenamiento continuaba disminuyendo.

Tampoco se identificó underfitting, ya que ambas pérdidas disminuyeron de manera constante a lo largo del entrenamiento. De hecho, las curvas seguían descendiendo al finalizar la época 100, lo que indica que el modelo aún tenía capacidad para seguir aprendiendo.

Para reducir el riesgo de sobreajuste se utilizó Dropout (0.2) en las capas ocultas y, en algunas configuraciones, regularización L2, la cual produjo una mejora en el desempeño de validación.

3. ¿La regularización (L1, L2 o dropout) mejoró el desempeño en validación? ¿Cuál funcionó mejor para este dataset y por qué cree que fue así?

Durante los experimentos se utilizó Dropout en todas las configuraciones y se evaluó el efecto de la regularización L2 mediante `weight_decay`. Los resultados mostraron que las configuraciones con regularización L2 obtuvieron un desempeño superior.

La mejor configuración del estudio combinó Dropout de 0.2 con regularización L2, ambas técnicas ayudaron a controlar la complejidad del modelo y favorecieron una mejor capacidad de generalización.

4. ¿Cómo se relacionan el batch size y el número de epochs con la estabilidad y velocidad del entrenamiento que observó?

En los experimentos se usó un batch size de 64 para el modelo, que proporcionó un entrenamiento estable y permitió que las pérdidas de entrenamiento y validación disminuyeran de forma suave durante las 100 épocas.

Respecto al número de épocas, se observó que el modelo continuaba mejorando al finalizar la época 100, por lo que un mayor número de épocas podría reducir el error. Sin embargo, aumentar las épocas también incrementa el tiempo de entrenamiento, por lo que se podría usar Early Stopping para detener el entrenamiento cuando la pérdida de validación deje de mejorar.

5. Comparando MSE, MAE y RMSE, ¿qué le dice cada métrica sobre el comportamiento de su modelo que las otras no muestran?

- **MSE** penaliza de forma severa los errores grandes, ya que eleva al cuadrado el error. Es útil para detectar predicciones con errores muy elevados.
- **MAE** representa el error absoluto promedio de las predicciones y es fácil de interpretar, porque se expresa en las mismas unidades que la variable objetivo. En este caso, indica que las predicciones del modelo presentan un error promedio cercano a 149 mil dólares.
- **RMSE** se expresa en las mismas unidades de la variable objetivo, pero mantiene la sensibilidad del MSE frente a errores grandes. Por esto, se usa como la principal métrica de comparación en regresiones.

6. Si tuviera que entrenar un modelo de producción con este dataset, ¿qué arquitectura e hiperparámetros elegiría, y qué estrategia de búsqueda (grid, random o bayesiana) usaría para seguir optimizando? Justifique.

Elegiría la configuración que obtuvo el mejor desempeño durante la validación, una arquitectura de dos capas ocultas con 128 y 64 neuronas, función de activación ReLU, optimizador Adam, tasa de aprendizaje de 0.001, Dropout de 0.2, regularización L2, batch size de 64 y 100 épocas de entrenamiento.

Para continuar optimizando el modelo utilizaría una búsqueda bayesiana. Esta usa la información obtenida en iteraciones anteriores para seleccionar las siguientes configuraciones a evaluar. Esto permite encontrar buenas combinaciones de hiperparámetros con menos entrenamientos, reduciendo el costo computacional.

5. Conclusiones

La mejor configuración del MLP estuvo compuesta por dos capas ocultas de 128 y 64 neuronas, activación ReLU, optimizador Adam y regularización L2, obteniendo un RMSE de 167,667.52 en validación y 169,401.10 en el conjunto de prueba.

Las curvas de pérdida mostraron un entrenamiento estable y sin evidencias de sobreajuste, ya que las pérdidas de entrenamiento y validación disminuyeron de forma similar durante las 100 épocas.

Los resultados evidenciaron que la selección de hiperparámetros influye en el desempeño del modelo. En particular, el optimizador Adam y la regularización L2 ofrecieron el mejor rendimiento.